CS3212 Algorithms

Aya Zirikly
10/22/2026

Rabin-Karp takeaways

- Simple: $O(1)$ arithmetic per shift, no tables, $O(1)$ extra space
- Expected $O(n + m)$ when hits are few
- Worst case $\Theta((n - m + 1) \cdot m)$, same as brute force
- Large prime $q \rightarrow$ few spurious hits ($\approx n/q$)

Rabin-Karp: many patterns at once

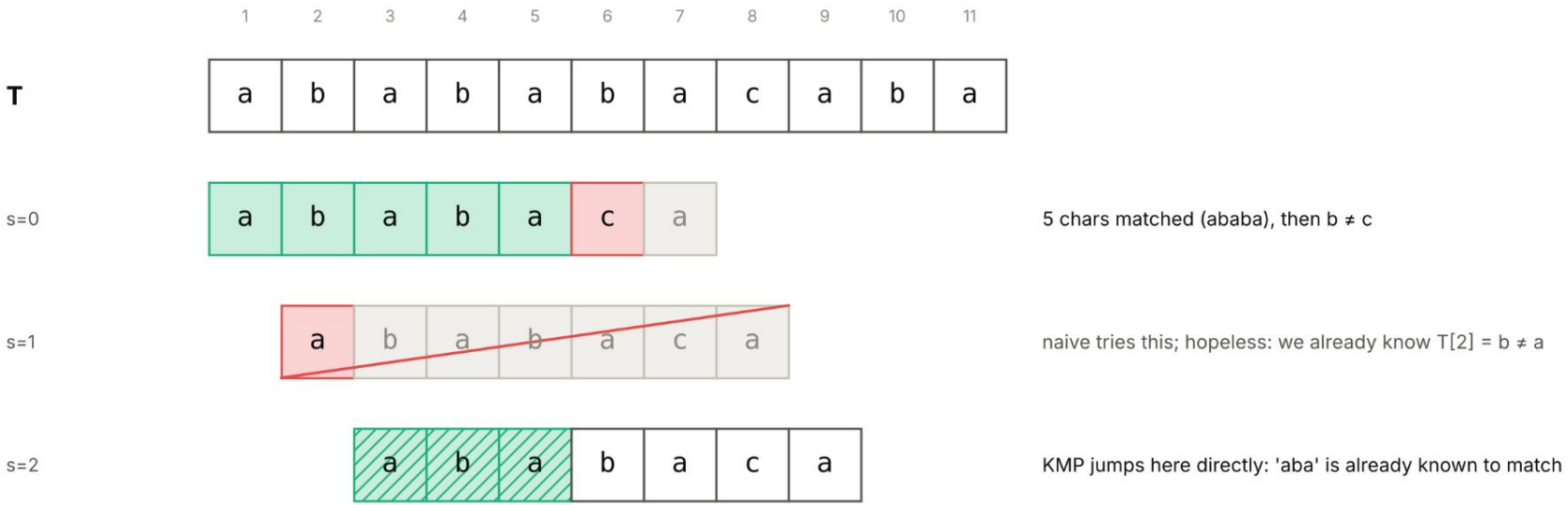
k patterns, same length m, one pass over T:

1. Hash each pattern into a set: {31415, 92653} → {7, 2} (q = 13)
2. Roll the hash over T once
3. Each shift: is t_s in the set? O(1)
4. Hit → verify against the pattern(s) with that hash

Knuth-Morris-Pratt (KMP)

KMP Motivation

- Brute force: after q letters match and the next one fails, it shifts P by 1 and re-reads those letters of T.
- Wasted work: the q letters it re-reads are exactly P[1..q], so we already know what they are.
- Idea: the next shift worth trying depends only on P, not on T, so we can compute it once in advance.
- Goal: the pointer into T never moves backward, so matching takes $\Theta(n)$.



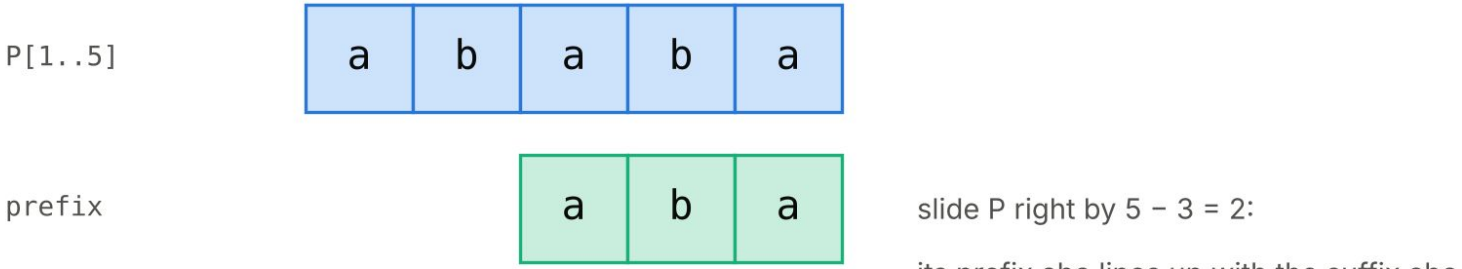
Key idea: the matched text ababa is part of P itself, so the next possible shift depends only on P.

KMP precomputes it: longest proper prefix of ababa that is also a suffix = aba (length 3). Text pointer never moves back.

The prefix function π

- **π[q]** = the length of the longest string that is both a prefix and a suffix of P[1..q], not counting P[1..q] itself
- Example, P = ababaca, q = 5: P[1..5] = ababa, which starts and ends with **aba**, so π[5] = 3
- Not counting the whole string means π[q] < q, so KMP always moves forward
- Use: after q letters match and the next one fails, shift P so its first π[q] letters sit under the last π[q] letters of T read. Those are already known to match.

π[5] for P = ababaca: P[1..5] = ababa

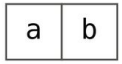
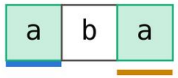
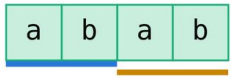
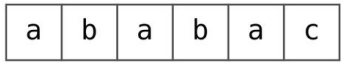
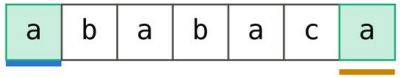


slide P right by 5 − 3 = 2:

its prefix aba lines up with the suffix aba

π[q] = length of the longest PROPER prefix of P[1..q] that is also a suffix of P[1..q]

`candidates for ababa:  a ✓ (len 1),  ab ✗,  aba ✓ (len 3),  abab ✗  →  π[5] = 3`

q	P[1..q]	longest prefix = suffix	π[q]
1		none (nothing shorter to compare)	0
2		none ('a' ≠ 'b')	0
3		a ('a' = 'a')	1
4		ab ('ab' = 'ab')	2
5		aba ('aba' = 'aba', sharing the middle a is fine)	3
6		none (no prefix ends in 'c')	0
7		a ('a' = 'a')	1

blue bar = the prefix, orange bar = the matching suffix, π[q] = their length (must be shorter than q)

Computing π (CLRS)

```
COMPUTE-PREFIX-FUNCTION(P)
  m = P.length
  let pi[1..m] be a new array
  pi[1] = 0
  k = 0
  for q = 2 to m
      while k > 0 and P[k+1] != P[q]
          k = pi[k]                  // fall back to the next shorter border
      if P[k+1] == P[q]
          k = k + 1
      pi[q] = k
  return pi
```

q	P[q]	k	line executed	k after	π[q]
2	b	0	while: k>0? no → skip		
			if P[1]=a == b? no → k unchanged	0	0
3	a	0	while: skip		
			if P[1]=a == a? yes → k = k+1	1	1
4	b	1	while: k>0, P[2]=b != b? no → exit		
			if P[2]=b == b? yes → k = k+1	2	2
5	a	2	while: P[3]=a != a? no → exit		
			if P[3]=a == a? yes → k = k+1	3	3
6	c	3	while: P[4]=b != c? yes → k = π[3]	1	
			while: P[2]=b != c? yes → k = π[1]	0	
			while: k>0? no → exit		
			if P[1]=a == c? no → k unchanged	0	0
7	a	0	while: skip		
			if P[1]=a == a? yes → k = k+1	1	1

Computing π (CLRS)

```
COMPUTE-PREFIX-FUNCTION(P)
  m = P.length
  let pi[1..m] be a new array
  pi[1] = 0
  k = 0
  for q = 2 to m
      while k > 0 and P[k+1] != P[q]
          k = pi[k]                  // fall back to the next shorter border
      if P[k+1] == P[q]
          k = k + 1
      pi[q] = k
  return pi
```

Time complexity?

Computing π (CLRS)

```
COMPUTE-PREFIX-FUNCTION(P)
  m = P.length
  let pi[1..m] be a new array
  pi[1] = 0
  k = 0
  for q = 2 to m
      while k > 0 and P[k+1] != P[q]
          k = pi[k]                  // fall back to the next shorter border
      if P[k+1] == P[q]
          k = k + 1
      pi[q] = k
  return pi
```

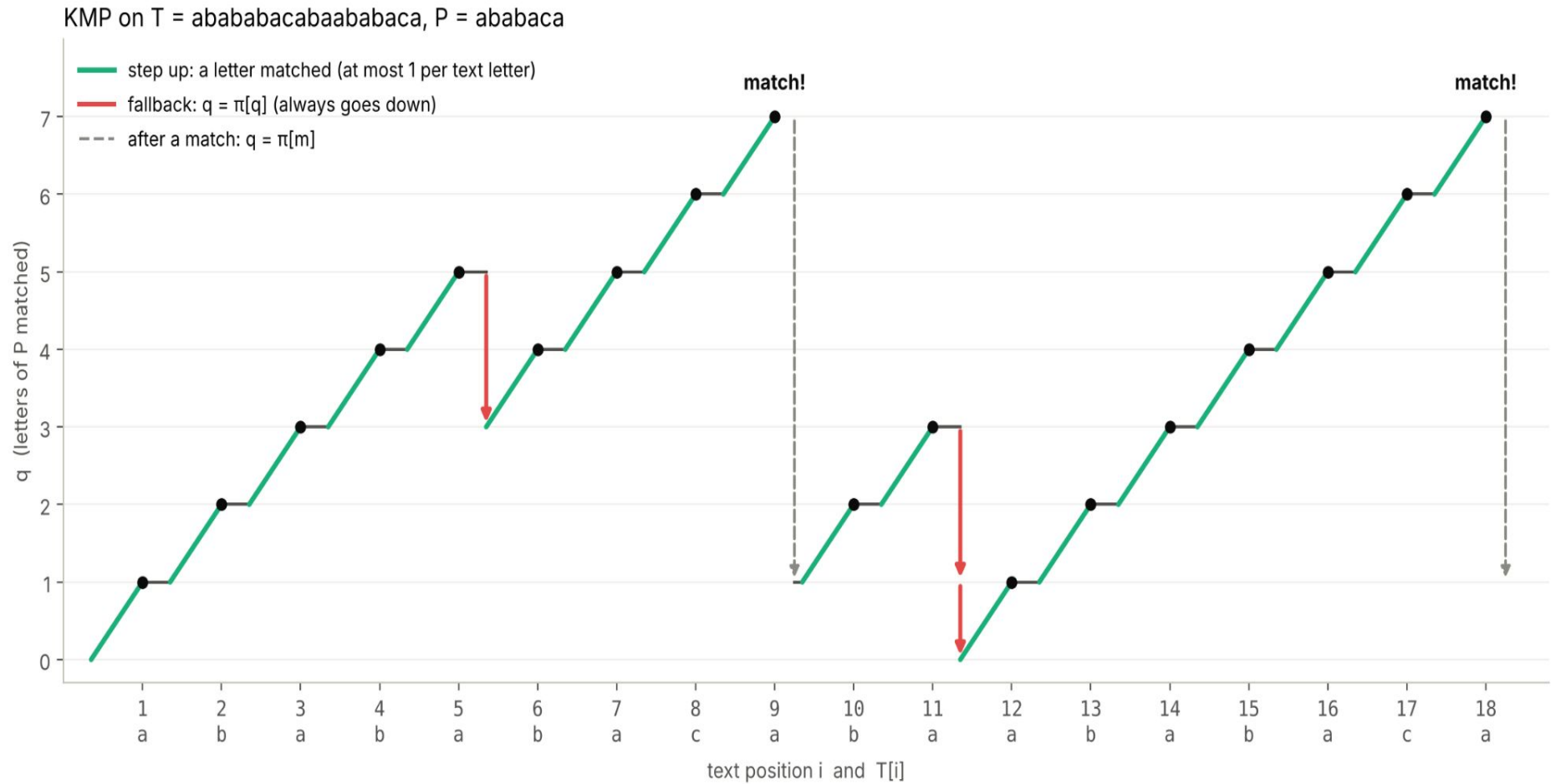
Time Θ(m)

The KMP matcher (CLRS)

```
KMP-MATCHER(T, P)
  n = T.length
  m = P.length
  pi = COMPUTE-PREFIX-FUNCTION(P)
  q = 0                          // number of characters matched
  for i = 1 to n                  // scan the text left to right, never back up
      while q > 0 and P[q+1] != T[i]
          q = pi[q]               // next character does not match
      if P[q+1] == T[i]
          q = q + 1               // next character matches
      if q == m                   // all of P matched
          print "Pattern occurs with shift" i - m
          q = pi[q]               // look for the next match
```

KMP on T = abababacaba, P = ababaca (π = 0 0 1 2 3 0 1)

i	T[i]	what happens	q after
1	a	P[1]=a matches: q = 1	1
2	b	P[2]=b matches: q = 2	2
3	a	P[3]=a matches: q = 3	3
4	b	P[4]=b matches: q = 4	4
5	a	P[5]=a matches: q = 5	5
6	b	P[6]=c ≠ b: q = π[5] = 3; P[4]=b matches: q = 4	4
7	a	P[5]=a matches: q = 5	5
8	c	P[6]=c matches: q = 6	6
9	a	P[7]=a matches: q = 7; q = m → MATCH at shift 2; q = π[7] = 1	1
10	b	P[2]=b matches: q = 2	2
11	a	P[3]=a matches: q = 3	3



n = 18 letters → at most 18 steps up → at most 18 fallbacks in total. Here: 3 fallbacks, comparisons = 18 + 3 = 21 ≤ 2n = 36

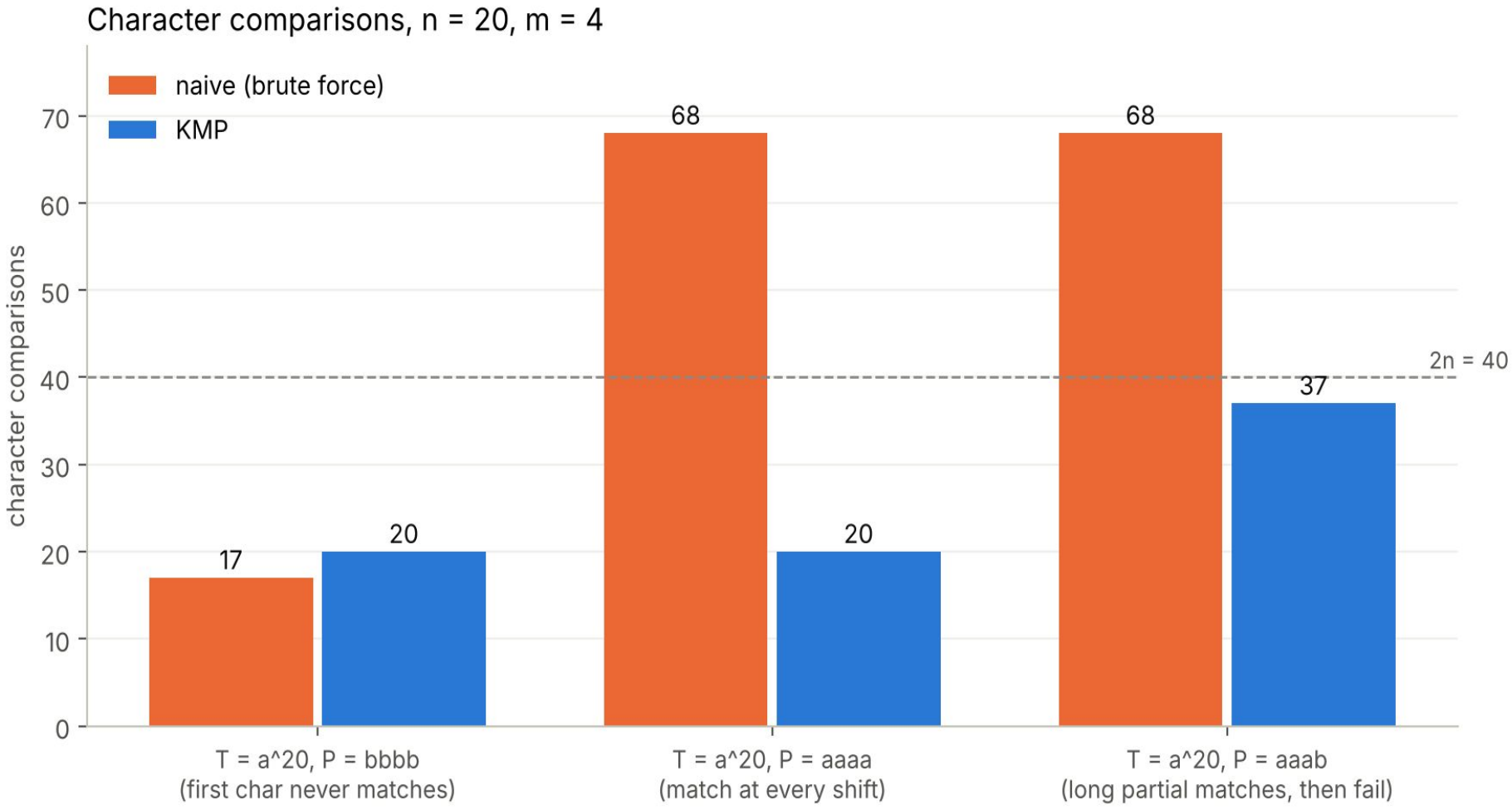
KMP Running Time

- Comparisons = n + #fallbacks (1 per letter + 1 per fallback)
  - q goes up ≤ 1 per letter; each fallback lowers q; q ≥ 0

    → #fallbacks ≤ n → at most 2n comparisons → $\Theta(n)$
- Computing $\pi \rightarrow \Theta(m)$
- Total $\Theta(n + m)$, best and worst case

KMP Best vs. Worst Case

- T = a^20 (n = 20, m = 4)
- Best: P = bbbb → 20 comparisons (= n)
- Worst: P = aaab → 37 (20 + 17 falls back) comparisons (< 2n = 40)
- Naive on aaab: 68 = (n − m + 1)·m


Character comparisons, n = 20, m = 4
naive (brute force)
KMP
character comparisons
0
10
20
30
40
50
60
70
17
20
68
20
68
37
2n = 40
T = a^20, P = bbbb
(first char never matches)
T = a^20, P = aaaa
(match at every shift)
T = a^20, P = aaab
(long partial matches, then fail)

Check your understanding (KMP running time)

- T = ababaababcabab, P = ababc (π = 0 0 1 2 0): # fallbacks? # comparisons?
- Text of length 8 where KMP does exactly 8 comparisons for P = ab?
- True or false: "KMP never spends more than 2 comparisons on one text letter."

KMP takeaways

- Never backs up in T → works on streams
- Guaranteed $\Theta(n + m)$
- Extra space $\Theta(m)$, independent of alphabet

Summary

Algorithm	Preprocessing	Matching: best	Matching: worst	Worst case caused by
Brute force	none	$\Theta(n - m + 1)$	$\Theta((n - m + 1) \cdot m)$	many near-matches, e.g. T = a^n, P = a^m
Rabin-Karp	$\Theta(m)$	$\Theta(n - m + 1)$	$\Theta((n - m + 1) \cdot m)$	every window hashes to p, e.g. T = a^n, P = a^m
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(n)$	no bad input: at most 2n comparisons

Graph Foundations & Traversals

Introduction to Graph Representations

Graph components: A graph G consists of a set of vertices (V) and edges (E).

Storage goal: Store connections efficiently in memory while supporting fast query and traversal operations.

Primary formats:

- **Adjacency Matrix:** A 2D grid mapping all possible vertex pair connections.
- **Adjacency List:** An array of dynamic collections listing only existing connections.

Key decision factor: Graph Density (Sparse graphs where $E \ll V^2$ and $E \approx V^2$).

Adjacency Matrix

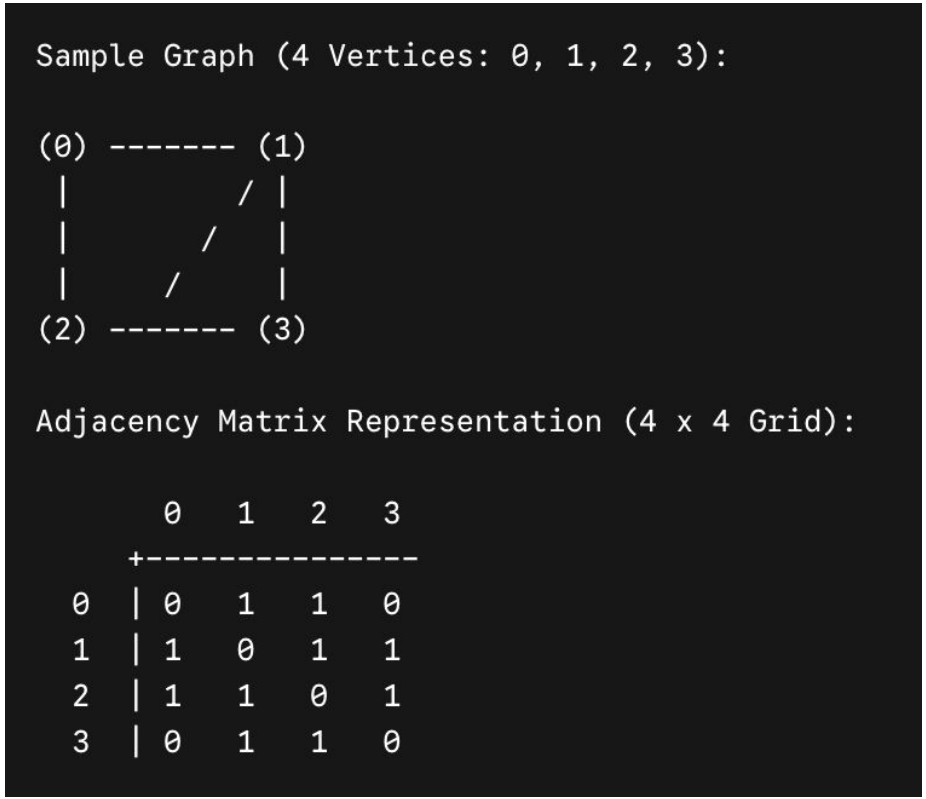
Definition & Properties

- A 2D array of size V x V where entry `matrix[i][j]` represents an edge from vertex `i` to vertex `j`.
- **Unweighted Graph:** `1` indicates an edge exists; `0` indicates no edge.
- **Weighted Graph:** Stores the edge weight (or `infinity`/`null` if no edge exists).
- **Undirected Graph:** Symmetrical across the main diagonal (`matrix[i][j] == matrix[j][i]`).

Adjacency Matrix

Definition & Properties

- A 2D array of size V x V where entry `matrix[i][j]` represents an edge from vertex `i` to vertex `j`.
- **Unweighted Graph:** `1` indicates an edge exists; `0` indicates no edge.
- **Weighted Graph:** Stores the edge weight (or `infinity`/`null` if no edge exists).
- **Undirected Graph:** Symmetrical across the main diagonal (`matrix[i][j] == matrix[j][i]`).



Trade-offs

- **Pros:**
  - Constant-time edge lookup: ?
  - Constant-time edge insertion/deletion: ?
  - Simple implementation using standard multi-dimensional arrays.
- **Cons?**

Trade-offs

- **Pros:**
  - Constant-time edge lookup: O(1) to check if edge (u, v) exists.
  - Constant-time edge insertion/deletion: O(1).
  - Simple implementation using standard multi-dimensional arrays.
- **Cons:**
  - High space overhead: Always consumes O(V^2) memory regardless of edge count.
  - Slow neighbor traversal: Iterating over all adjacent nodes for a vertex requires scanning an entire row of length O(V).

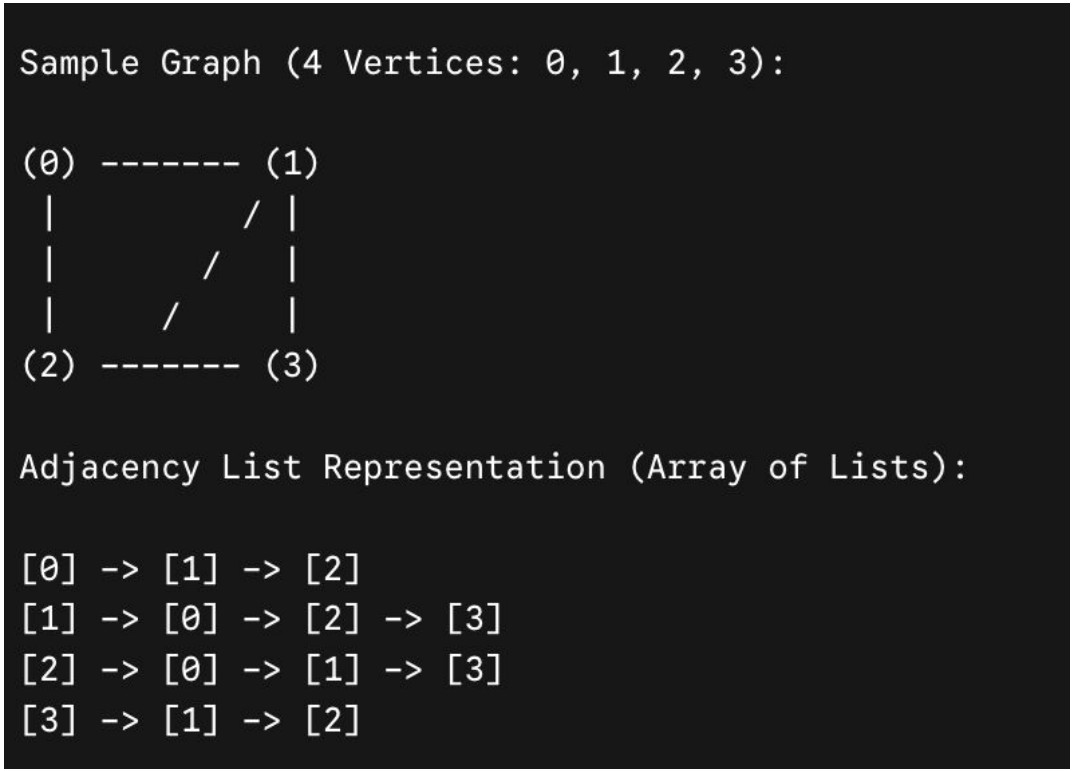
Adjacency List

Definition & Properties

- An array of size V, where index `i` stores a linked list, vector, or dynamic array of all vertices directly connected to vertex `i`.
- Degree of vertex u, degree(u): The number of vertices adjacent to vertex u.
- **Unweighted Graph:** Nodes in the list contain target vertex IDs.
- **Weighted Graph:** Nodes store tuples of `(target_vertex, weight)`.

Adjacency List

An array of size V, where index `i` stores a linked list, vector, or dynamic array of all vertices directly connected to vertex `i`.



Trade-offs

Pros:

- Memory efficient: Requires O(V + E) space; ideal for sparse real-world networks.
- Fast neighbor iteration: Traversing neighbors takes O(degree(u)) time instead of scanning all V nodes.
- Dynamic insertion: Appending a new edge takes O(1) time.
- Efficient for sparse graphs |E| << |V|^2

Cons:

- Slower edge lookup: Checking if edge (u, v) exists requires searching vertex u's list in O(degree(u)) time.

Memory Layout Comparison

```
Graph: 4 Vertices, 2 Edges (0-1, 1-2)  [Sparse Graph]

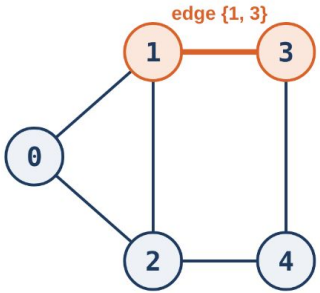
ADJACENCY MATRIX (Fixed O(V^2) Storage):
       0   1   2   3
  0 [  0   1   0   0  ]  <- 16 memory slots allocated
  1 [  1   0   1   0  ]     to store only 4 directed connections.
  2 [  0   1   0   0  ]     Wastes memory on 0s.
  3 [  0   0   0   0  ]

ADJACENCY LIST (Dynamic O(V + E) Storage):
  [0] -> [1]
  [1] -> [0] -> [2]      <- Only allocates memory for
  [2] -> [1]             <- actual existing edges.
  [3] -> (empty)
```

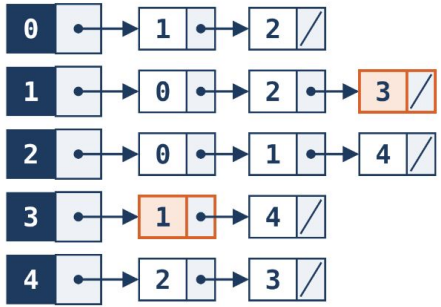
Adjacency List Representation

Undirected, unweighted graph G = (V, E) with |V| = 5, |E| = 6

Graph G



Adjacency list: Adj[0..4]



/ marks the end of a list (NIL)

Adjacency matrix A

	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	0
2	1	1	0	0	1
3	0	1	0	0	1
4	0	0	1	1	0

Symmetric: A[u][v] = A[v][u] = 1

Each undirected edge is stored twice

Edge {1, 3} appears as 3 in Adj[1]
and as 1 in Adj[3].
Total list entries = Σ deg(v) = 2|E| = 12
(For a directed graph: |E| entries, once each.)

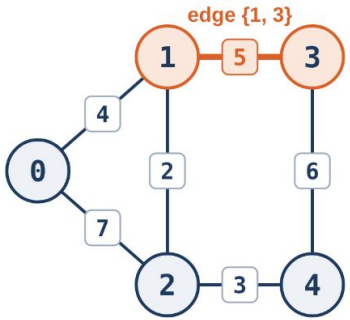
	Adjacency list	Matrix
Space	**V + 2E = 17 → Θ(V + E)**	V² = 25 → Θ(V²)
List all neighbors of u	**Θ(deg u)**	Θ(V)
Is {u, v} an edge?	**O(deg u)**	Θ(1)
BFS / DFS on whole graph	**Θ(V + E)**	Θ(V²)

Sparse graphs (E ≪ V²): the list wins on space and on traversal time.

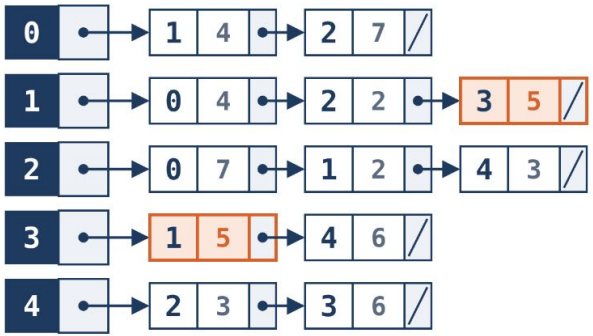
Adjacency List Representation: Weighted

Undirected graph G = (V, E) with weight function w : E → ℝ, |V| = 5, |E| = 6

Graph G



Adjacency list: Adj[0..4]



Node = (neighbor v, weight w(u, v), next) · ⁄ marks the end of a list (NIL)

Adjacency matrix A

	0	1	2	3	4
0	∞	4	7	∞	∞
1	4	∞	2	5	∞
2	7	2	∞	∞	3
3	∞	5	∞	∞	6
4	∞	∞	3	6	∞

A[u][v] = w(u, v); ∞ = no edge. Symmetric.

Each edge is still stored twice

Edge {1, 3} with w = 5 appears as (3, 5) in Adj[1] and as (1, 5) in Adj[3].

Entries = 2|E| = 12, each one word larger.

Weight costs O(1) extra per entry, so space stays Θ(V + E).

	Adjacency list	Matrix
Space	**V + 2E = 17 → Θ(V + E)**	$V^2 = 25 \rightarrow \Theta(V^2)$
Visit neighbors of u with weights	**Θ(deg u)**	Θ(V)
Look up w(u, v)	**O(deg u)**	Θ(1)
Dijkstra (binary heap)	**O((V + E) log V)**	$\Theta(V^2)$

Sparse graphs ($E \ll V^2$): the list wins on space and on traversal time.

Complexity Summary Table

Operation / Metric	Adjacency Matrix	Adjacency List
Space Complexity	O(V^2)	O(V + E)
Check Edge Existence (u, v)	O(1)	O(degree(u))
Add Edge	O(1)	O(1)
Remove Edge	O(1)	O(degree(u))
Find All Neighbors of u	O(V)	O(degree(u))
Add a New Vertex	O(V^2) (Requires grid resize)	O(1) (Append to array)

Selection Matrix

Choose Adjacency Matrix When:

- The graph is **Dense** (E is close to V^2).
- The primary workload relies on frequent edge existence queries (e.g., `hasEdge(u, v)`).
- The total vertex count V is small and static.
- Matrix-based algorithms are required (e.g., Floyd-Warshall, spectral graph methods).

Choose Adjacency List When:

- The graph is **Sparse** (E is significantly smaller than V^2; typical in social networks, web links, maps).
- Graph traversal algorithms are used (e.g., BFS, DFS, Dijkstra, Topological Sort).
- Memory efficiency is a priority.
- Vertices and edges need to be added or modified dynamically.